

Ansible – Twenty CRM Automation

Objective

Automate the complete setup of Twenty CRM (an open-source CRM application) on a Linux target host using Ansible — including dependency installation, Docker setup, a dedicated application user, directory/permission management, source control checkout, templated configuration, container orchestration via Docker Compose, a change-triggered restart handler, and post-deployment verification.

Environment Used

The playbook was developed and tested using the **Killercoda Ubuntu 24.04 playground**, with the Ansible control node and the managed target node communicating over SSH. (Initial development started on the KodeKloud Ansible sandbox; the exercise was continued and completed on Killercoda due to KodeKloud session/quota limits.)

Project Structure

- `ansible.cfg` — Ansible configuration: sets the default inventory path, disables SSH host-key prompts for the lab environment, and enables privilege escalation (become) by default.
- `inventory/hosts.ini` — Defines the `twenty_crm` host group containing the target node's address, SSH user, and the Python interpreter path used on the target.
- `group_vars/all.yml` — Centralised variables for the whole play: the application system user/group, install directories, Git repository details, Docker image tag, application port, PostgreSQL credentials, and the app secret/server URL used to render the configuration templates.
- `templates/env.j2` — Jinja2 template that generates the application's `.env` file from the variables in `group_vars/all.yml` (database connection string, server/frontend URLs, app secret, storage and Redis settings).
- `templates/docker-compose.yml.j2` — Jinja2 template that generates the Docker Compose file defining the four services that make up the Twenty CRM stack: PostgreSQL (db), Redis (redis), the API server (server), and the background job processor (worker).
- `playbook.yml` — The main Ansible playbook containing all tasks and the restart handler.

Process Followed

1. **Ansible control node setup** — Installed Ansible (via the official Ansible PPA) and sshpass/SSH key-based auth on the control node so it could reach the target node non-interactively.
2. **SSH connectivity** — Generated an SSH key pair on the control node and copied the public key to the target node's root account so Ansible could connect without prompting for a password on every run. Verified connectivity with `ansible <group> -m ping` before running the playbook.
3. **Inventory creation** — Defined the target host under a `twenty_crm` group in `inventory/hosts.ini`, along with the Python interpreter path required for Ansible modules to run correctly on the target.
4. **Dependency installation** — The playbook updates the apt cache and installs baseline packages required for the rest of the automation: `ca-certificates`, `curl`, `gnupg`, `lsb-release`, `git`, and `python3-pip`.
5. **Docker installation** — Docker's official GPG key is downloaded into `/etc/apt/keyrings`, the official Docker apt repository is registered using that key, and Docker Engine, the CLI, `containerd`, `Buildx`, and the Compose plugin are installed. The Docker Python SDK (`python3-docker`) is installed via apt (not pip) because Ubuntu 24.04 enforces PEP 668's externally-managed-environment restriction, which blocks system-wide pip install. The Docker service is then enabled and started via `systemd`.
6. **Dedicated application user** — A dedicated system group and user (`twenty`) are created for running the application, added to the `docker` group so it can manage containers without full root privileges, and its Ansible remote temp directory is pre-created with correct ownership to avoid permission warnings when tasks run as that user.
7. **Directories and permissions** — Two directories are created and owned by the `twenty` user/group: one to hold the Docker Compose project (`.env` and `docker-compose.yml`), and a separate one to hold the cloned source repository, keeping the two concerns cleanly separated.
8. **Cloning the repository** — The official Twenty CRM GitHub repository is cloned (shallow clone, depth: 1) into the repository directory, running as the dedicated `twenty` user rather than root.
9. **Configuration via Jinja2 templates** — Rather than relying on any pre-existing compose file bundled in the cloned repository (which is not guaranteed to be stable or self-hosting-ready), the playbook generates its own configuration from templates:

- `env.j2` renders the `.env` file with database credentials, secrets, and URLs pulled from `group_vars/all.yml`.
 - `docker-compose.yml.j2` renders a Compose file wiring together PostgreSQL, Redis, the Twenty server, and the Twenty worker using the official `twentycrm/twenty-server` image. Both templating tasks notify the restart handler, so any future change to variables or templates automatically triggers a container restart.
10. **Starting the application** — Docker Compose is invoked via the `community.docker.docker_compose_v2` module against the rendered project directory to bring up all four services, running as the `twenty` user.
 11. **Restart handler** — A single handler (`restart twenty crm`) uses the same Compose module with `state: restarted` and is only triggered when the `.env` or `docker-compose.yml` templates actually change content — it does not run on every playbook execution, which keeps the automation efficient and idempotent.
 12. **Verification** — The playbook waits for the application's port to become reachable, then repeatedly polls the backend's `/healthz` endpoint until it returns HTTP 200 (with retries and delay to allow the containers time to fully initialise), and prints the final verified status.
 13. **Idempotency testing** — The playbook was executed twice in succession. On the second run, package installation, user/directory creation, and templating tasks reported no changes, and the restart handler did not fire (since the rendered file content was identical), confirming the playbook is safe to re-run without unwanted side effects.
 14. **Manual verification on the target node** — After the playbook run, the deployment was confirmed directly on the target using `docker ps` (to confirm all four containers were healthy/running) and `curl` against the application's port and health endpoint.

Walkthrough of `playbook.yml`

The playbook targets the `twenty_crm` host group defined in the inventory, runs with `become: true` so every task can escalate to root when needed, and loads all variables from `group_vars/all.yml` via `vars_files`. Below is what each section of the play does, in the order it executes.

Handler block (`restart twenty crm`) Defined at the top of the play, before the tasks, this handler is not run automatically — it only fires when a task later in the play explicitly notifies it. It calls the Docker Compose module with `state: restarted` against the compose project directory, executed as the dedicated `twenty` user. Its only job is to bounce the containers when configuration actually changes.

1. Dependency installation (two tasks) The play first refreshes the apt package cache (with a one-hour cache validity window so repeated runs don't re-download package indexes unnecessarily), then installs the baseline packages the rest of the playbook depends on: ca-certificates and gnupg for verifying the Docker repository's signing key, curl and lsb-release for the download and distribution-detection steps that follow, git for cloning the Twenty CRM source, and python3-pip as a general-purpose Python packaging tool.

2. Docker installation (seven tasks) This block installs Docker Engine using Docker's official apt repository rather than the distribution's own (older) Docker packages:

- Creates /etc/apt/keyrings to hold the repository's signing key.
- Downloads Docker's GPG key into that directory.
- Detects the system's CPU architecture with dpkg --print-architecture, storing it for use in the repository definition (this task is marked as never "changed" since it only reads system information).
- Registers Docker's official apt repository, scoped to the detected architecture and the host's Ubuntu release codename, signed by the downloaded key.
- Installs the Docker Engine, CLI, containerd, Buildx plugin, and Compose v2 plugin as a single apt task.
- Installs python3-docker, the Docker SDK for Python, via apt rather than pip — this is required for Ansible's community.docker.docker_compose_v2 module to function, and using apt avoids Ubuntu 24.04's PEP 668 restriction on system-wide pip installs.
- Ensures the docker systemd service is both started and enabled on boot.

3. Application user creation (three tasks)

- Creates a dedicated twenty system group.
- Creates a dedicated twenty user belonging to that group, additionally added to the docker group (so it can run Docker commands without needing root), with a real shell and home directory.
- Pre-creates that user's Ansible remote-temp directory with the correct ownership and permissions, avoiding the warning Ansible otherwise prints the first time a task runs as a non-default remote user.

4. Directories and permissions (two tasks) Creates two separate directories, both owned by the twenty user/group: one for the Docker Compose project (where the rendered .env and docker-compose.yml will live), and a second, distinct directory that will hold the cloned application source code. Keeping these separate means the

generated configuration is never at risk of being overwritten by a future git pull or re-clone.

5. Cloning the repository (one task) Uses the git module to shallow-clone (depth: 1) the official Twenty CRM GitHub repository at the branch/tag specified by the `repo_version` variable into the source directory created above. This task runs as the `twenty` user rather than `root`, so the checked-out files are owned by the same account that will later run the containers.

6. Configuration via Jinja2 templates (two tasks)

- Renders `templates/env.j2` to `.env` inside the compose project directory, substituting in the Postgres credentials, application secret, server/frontend URLs, and other values from `group_vars/all.yml`. File permissions are locked to `0600` since it contains secrets.
- Renders `templates/docker-compose.yml.j2` to `docker-compose.yml` in the same directory, defining the four services that make up the stack. Both tasks notify: `restart twenty crm` — Ansible only actually calls the handler if the rendered file content differs from what's already on disk, which is what makes this pair of tasks (and the handler) idempotent.

7. Starting the application (one task) Runs `community.docker.docker_compose_v2` with `state: present` and `pull: missing` against the compose project directory, as the `twenty` user. This brings up all services defined in the rendered `docker-compose.yml` (pulling images only if they aren't already present locally), and is safe to re-run — Compose itself will not recreate containers that already match the current configuration.

8. Verification (three tasks)

- `wait_for` blocks until the application's TCP port is accepting connections, with a generous timeout to allow for slow image pulls or first-time database initialisation.
- A `uri` task then repeatedly polls the backend's `/healthz` endpoint (rather than the root path, which the Twenty API server correctly returns a 404 for) until it responds with HTTP 200, retrying with a delay between attempts to tolerate the short window while the server finishes starting up.
- A final debug task prints the confirmed HTTP status, giving a clear, human-readable confirmation in the playbook output that the deployment succeeded.

Key Design Decisions

- **Separate compose and source directories** — keeps the Ansible-managed configuration (`.env`, `docker-compose.yml`) independent from the cloned

application source, so re-cloning or updating the repo never risks overwriting generated configuration.

- **Self-authored Compose template instead of the repo's own file** — the official Twenty CRM images and self-hosting configuration change between versions, so generating the Compose file from an Ansible template (driven by variables) makes the deployment configuration explicit, version controlled, and consistent with the task's requirement to configure the application via Ansible variables and Jinja2 templates.
- **apt over pip for the Docker Python SDK** — avoids fighting Ubuntu 24.04's externally-managed-environment protection and keeps the task idempotent and dependency-light.
- **Health endpoint over root path for verification** — the Twenty CRM backend server returns a 404 JSON response at / (it is an API service, not a page-serving frontend), so verification checks the documented /healthz endpoint instead.